Cold Storage Monitoring System

with FastAPI, Role-Based Access Control, and Automated Sensor Simulation

---

Technical Project Report

A web-based platform for monitoring warehouses, storage units, sensors, environmental telemetry, alerts, analytics, and user-specific access.

Submitted By: _____

Roll No. / ID: _____

Department: _____

Institution: _____

Guide / Mentor: _____

Technology Stack: Python, FastAPI, SQLAlchemy, SQLite, HTML, CSS, Jinja2, Uvicorn

Date: June 10, 2026

Contents

1 Executive Summary

The **Cold Storage Monitoring System** is a web-based software application developed to monitor cold storage warehouses used for storing temperature-sensitive medicines, vaccines, and pharmaceutical products. The system manages warehouses, storage units, sensors, sensor-generated data, alerts, and alert-resolution activities.

The backend is implemented using **FastAPI** and **SQLAlchemy**, while the frontend uses **HTML**, **CSS**, and **Jinja2 templates**. The application supports two major user roles: **Admin** and **User**. The Admin can manage users, assign warehouses, view warehouse counts, monitor alert statistics, and view dashboard analytics. The User can register, log in, view assigned warehouses, create storage units, automatically generate default sensors, view sensor data, and resolve only their own alerts.

The project also includes an automated simulator module that generates virtual readings for temperature, humidity, and door status sensors. These readings are sent to the backend through the `/data/` endpoint. If readings violate configured rules, alerts are generated and displayed on dashboards.

Table 1: Project Summary

Item	Description
Project Title	Cold Storage Monitoring System
Backend	FastAPI with SQLAlchemy ORM
Frontend	HTML, CSS, and Jinja2 templates
Database	SQLite
User Roles	Admin and User
Sensor Types	Temperature, Humidity, Door Status
Main Features	Warehouse assignment, storage unit management, sensor simulation, alert generation, dashboards

2 System Architecture

The system follows a layered architecture. The frontend layer provides dashboard pages for Admin and User roles. The backend layer contains FastAPI routes for authentication, warehouse management, storage unit management, sensor data, alerts, and analytics. The service layer contains simulator logic, rule evaluation logic, and alert generation logic. The persistence layer stores all records in an SQLite database using SQLAlchemy models.

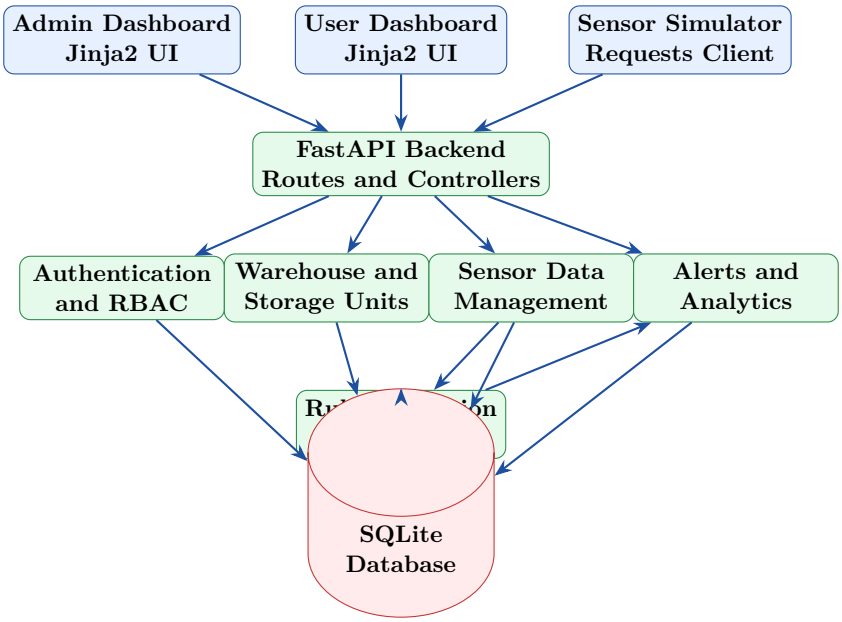

Figure 1: High-Level System Architecture

2.1 Directory Structure

The project is organized into separate modules for routes, database models, templates, static files, simulator logic, and configuration. A modular structure improves maintainability and makes the project easier to extend.

Listing 1: Project Directory Structure

```
cold-storage-monitoring/
|-- main.py
|-- database.py
|-- models.py
|-- schemas.py
|-- auth.py
|-- simulator.py
|-- alert_engine.py
|-- analytics.py
|-- requirements.txt
|-- routes/
|   |-- admin.py
|   |-- users.py
|   |-- warehouses.py
|   |-- storage_units.py
|   |-- sensors.py
|   |-- data.py
|   `-- alerts.py
|-- templates/
|   |-- login.html
```

```
|   |-- register.html
|   |-- admin_dashboard.html
|   |-- user_dashboard.html
|   |-- warehouses.html
|   |-- storage_units.html
|   |-- sensors.html
|   `-- alerts.html
|-- static/
|   |-- style.css
|   `-- charts.js
`-- tests/
    |-- test_auth.py
    |-- test_storage_units.py
    |-- test_alerts.py
    `-- test_simulator.py
```

3 Technology Stack

The project uses a lightweight and open-source technology stack suitable for academic development, local execution, and future deployment.

Table 2: Technology Stack

Layer	**Technology**	**Purpose**
Programming Language	Python	Backend programming and simulator development
Backend Framework	FastAPI	API development and web route handling
ORM	SQLAlchemy	Database modeling and query handling
Database	SQLite	Local persistent storage
Frontend	HTML, CSS	User interface layout and styling
Templates	Jinja2	Dynamic server-side rendering
Server	Uvicorn	Running FastAPI ASGI application
Simulator Communication	Requests	Sending generated sensor readings to backend API
Authentication	Cookies / Session-style login	Role-based access control
Testing	Pytest	Unit and functional testing

4 Database Design

The database stores users, warehouses, storage units, sensors, sensor readings, alerts, alert logs, and rules. SQLAlchemy models are used to define the database structure and relationships.

4.1 Entity–Relationship Overview

The database design follows a relational structure. A user may be assigned one or more warehouses. A warehouse can contain multiple storage units. A storage unit automatically contains three default sensors: temperature, humidity, and door status. Sensors generate sensor data. Sensor readings are checked against rules, and alerts are created when rule conditions are violated.

- **User** represents Admin or normal User accounts.
- **Warehouse** represents cold storage warehouses assigned to users.
- **StorageUnit** represents units such as cold rooms, freezers, or vaccine chambers.
- **Sensor** represents temperature, humidity, or door status sensor.
- **SensorData** stores generated sensor readings.
- **Alert** stores unsafe condition alerts.
- **AlertLog** stores alert action history.
- **Rule** stores threshold conditions.

4.2 Table Definitions

Table 3: Database Table Definitions

Table	Important Fields	Description
`users`	id, name, email, password, role	Stores Admin and User account details.
`warehouses`	id, name, location, user_id	Stores warehouses assigned to users.
`storage_units`	id, name, unit_type, warehouse_id	Stores storage unit details.
`sensors`	id, sensor_type, storage_unit_id, is_active	Stores sensor details.
`sensor_data`	id, sensor_id, value, timestamp	Stores generated sensor readings.

Table	Important Fields	Description
`alerts`	id, sensor_id, alert_type, message, status	Stores alerts generated by rule violations.
`alert_logs`	id, alert_id, action, timestamp	Stores alert resolution and action history.
`rules`	id, sensor_type, condition, threshold	Stores alert threshold rules.

4.3 Entity–Relationship Diagram

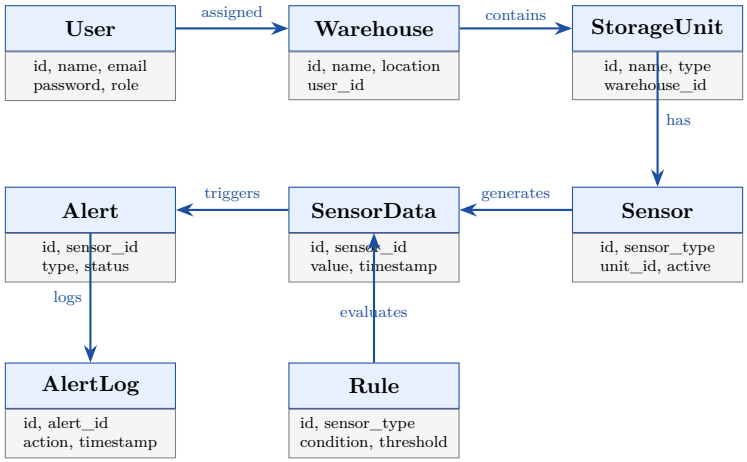

Figure 2: Entity–Relationship Diagram

5 API Reference

The backend exposes APIs and web routes for authentication, dashboards, warehouses, storage units, sensors, sensor data, alerts, and simulator operations.

Table 4: API Reference

Method	Endpoint	Description
GET	`/`	Health check or landing endpoint.
GET	`/login`	Displays login page.
POST	`/login`	Authenticates Admin or User.
GET	`/register`	Displays user registration page.
POST	`/register`	Creates a new user account.
GET	`/admin/dashboard`	Displays admin dashboard with users, warehouses, alerts, and analytics.

Method	Endpoint	Description
GET	`/user/dashboard`	Displays user dashboard with assigned warehouses and user alerts.
POST	`/warehouses/create`	Creates or assigns a warehouse to a user.
GET	`/warehouses`	Lists warehouses. Admin sees all; user sees assigned warehouses.
POST	`/warehouses/delete/{id}`	Deletes a warehouse and dependent records.
POST	`/storage-units/create`	Creates storage unit and automatically creates default sensors.
GET	`/storage-units`	Displays storage units for the logged-in user.
GET	`/sensors`	Displays sensors linked to user-owned storage units.
GET	`/sensors/active`	Returns active sensors for simulator usage.
POST	`/data/`	Receives simulator-generated sensor data and triggers alert evaluation.
GET	`/alerts`	Displays alerts according to role and ownership.
POST	`/alerts/resolve/{id}`	Allows a user to resolve only their own alert.
GET	`/logout`	Clears login session and logs out the user.

6 Authentication & Security

The system uses role-based access control to separate Admin and User operations. During login, the backend verifies credentials and identifies the role of the account. Based on the role, the user is redirected to the correct dashboard.

Admin security rules:

- Admin can view registered users.
- Admin can assign warehouses to users.
- Admin can view global alert analytics.
- Admin can view alerts but cannot resolve them.

User security rules:

- User can view only assigned warehouses.
- User can manage only their own storage units and sensors.
- User can view only alerts linked to their own storage units and sensors.
- User can resolve only their own alerts.

This design prevents unauthorized access to another user's warehouses, storage units, sensors, and alerts.

7 Alert Detection Engine

The alert detection engine evaluates every incoming sensor reading. The backend first stores the sensor data and then checks the value against predefined rules. If a rule condition is violated, an alert is created.

Table 5: Alert Detection Rules

Sensor Type	Condition	Generated Alert
Temperature	Value exceeds maximum threshold	Temperature exceeds maximum threshold
Humidity	Value exceeds maximum threshold	Humidity exceeds maximum threshold
Door Status	Door value is open	Door open alert

Listing 2: Sample Rule Evaluation Logic

```
def evaluate_sensor_data(sensor, value):
    if sensor.sensor_type == "temperature":
        if float(value) > 8:
            return "Temperature exceeds maximum threshold"

    elif sensor.sensor_type == "humidity":
        if float(value) > 60:
            return "Humidity exceeds maximum threshold"

    elif sensor.sensor_type == "door_status":
        if str(value).lower() == "open":
            return "Door is open"

    return None
```

8 Analytics Service

The analytics service prepares dashboard metrics for Admin and User dashboards. Admin analytics provide a system-wide view, while user analytics show only user-specific data.

Table 6: Dashboard Analytics Metrics

Metric	Description
Warehouse Count	Number of warehouses registered or assigned in the system.
Total Alerts	Total number of alerts generated.
Active Alerts	Alerts that are not yet resolved.
Solved Alerts	Alerts resolved by users.
Alert Type Distribution	Pie chart showing temperature, humidity, and door status alerts.
Alerts per Storage Unit	Bar graph showing alert count for each storage unit.
Registered Users	Number of users visible to Admin.

Analytics help identify frequently affected storage units and common alert causes. This supports faster decision-making and better monitoring of cold storage conditions.

9 Frontend Architecture

The frontend is implemented using HTML, CSS, and Jinja2 templates. Jinja2 allows dynamic rendering of database-driven content such as warehouse lists, storage units, sensor lists, alert tables, and dashboard statistics.

Main frontend pages:

- Login page
- Register page
- Admin dashboard
- User dashboard
- Warehouse assignment page
- Storage unit creation page
- Sensor list page
- Alerts table page

9.1 User Flow Diagrams

The user flow describes how Admin and User interact with the system. Admin logs in, assigns warehouses, views analytics, and monitors alerts. User registers, logs in, creates storage units, views sensors, and resolves own alerts.

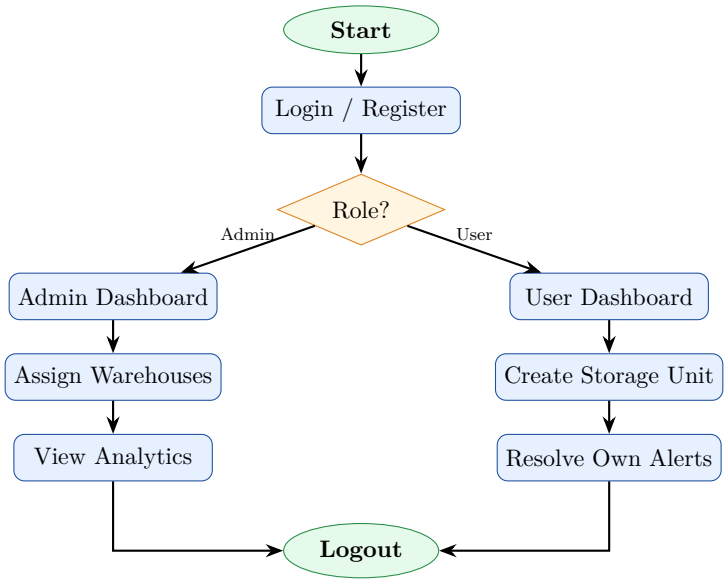

Figure 3: Admin and User Flow Diagram

10 Smart Sensor Simulator

The original simulator concept is similar to a smart meter simulator, but in this project it is adapted as a **smart sensor simulator** for cold storage monitoring. It automatically generates temperature, humidity, and door status readings.

The simulator performs the following operations:

1. Fetch active sensors from the backend.
2. Randomly select one sensor.
3. Generate a reading based on the sensor type.
4. Generate abnormal readings occasionally to trigger alerts.
5. Send generated readings to the backend `/data/` endpoint.
6. Repeat the process automatically while the backend is running.

Listing 3: Simplified Sensor Simulator Logic

```
import random
import requests
import time

BASE_URL = "http://127.0.0.1:8000"

def generate_value(sensor_type):
    if sensor_type == "temperature":
        return random.choice([random.uniform(2, 8), random.uniform
            (9, 15)])
    if sensor_type == "humidity":
        return random.choice([random.uniform(40, 60), random.uniform
```

```
            (65, 85)])
    if sensor_type == "door_status":
        return random.choice(["closed", "open"])

def simulator_loop():
    while True:
        sensors = requests.get(f"{BASE_URL}/sensors/active").json()
        if sensors:
            sensor = random.choice(sensors)
            value = generate_value(sensor["sensor_type"])
            requests.post(f"{BASE_URL}/data/", json={
                "sensor_id": sensor["id"],
                "value": value
            })
        time.sleep(5)
```

11 Testing Strategy

Testing verifies whether system modules work correctly and whether role-based restrictions are properly enforced.

Table 7: Test Cases

ID	Test Case	Action	Expected Result
TC01	User Registration	Enter valid user details	User account is created
TC02	Admin Login	Enter valid admin credentials	Admin dashboard opens
TC03	User Login	Enter valid user credentials	User dashboard opens
TC04	Warehouse Assignment	Admin assigns warehouse to user	Warehouse appears in user dashboard
TC05	Storage Unit Creation	User creates a storage unit	Default sensors are created
TC06	Temperature Alert	Simulator sends high temperature	Temperature alert is generated
TC07	Humidity Alert	Simulator sends high humidity	Humidity alert is generated
TC08	Door Alert	Simulator sends open door status	Door open alert is generated

ID	Test Case	Action	Expected Result
TC09	Alert Resolution	User resolves own alert	Alert status changes to solved
TC10	Unauthorized Resolution	User tries to resolve another user's alert	Action is denied

12 Deployment & Configuration

The application can be executed locally using Uvicorn. SQLite is used for local database storage. The simulator runs along with the backend or as a separate Python process.

Listing 4: Application Startup

```
# Install dependencies
pip install -r requirements.txt

# Run FastAPI application
uvicorn main:app --reload

# Open application
http://127.0.0.1:8000/login
```

Table 8: Configuration Parameters

Configuration	Description
Database URL	SQLite database path used by SQLAlchemy.
Admin Credentials	Default admin login details configured during setup.
Simulator Interval	Time delay between generated sensor readings.
Temperature Threshold	Maximum allowed temperature before alert generation.
Humidity Threshold	Maximum allowed humidity before alert generation.
Session/Cookie Settings	Used for session-style login and role tracking.

13 Billing Estimation Model

In this cold storage project, the billing estimation concept is adapted as an **operational risk and cost estimation model**. Instead of electricity billing, the model estimates potential risk cost based on unsafe storage conditions and alert severity.

A simple estimation model can be used for academic demonstration:

- Temperature alert = high risk cost
- Humidity alert = medium risk cost
- Door open alert = medium risk cost

- Repeated alerts for the same storage unit increase estimated risk

Table 9: Risk Cost Estimation Model

Alert Type	Risk Level	Estimated Impact
Temperature Alert	High	Product damage, regulatory risk, high financial loss
Humidity Alert	Medium	Product quality degradation and packaging risk
Door Open Alert	Medium	Cooling loss and increased temperature fluctuation
Repeated Alerts	Critical	Indicates operational failure or equipment issue

This model helps administrators identify storage units that may require urgent maintenance or operational review.

14 Conclusion

The Cold Storage Monitoring System provides a complete web-based solution for managing cold storage warehouses, storage units, sensors, sensor readings, alerts, and dashboard analytics. The project implements Admin and User role-based access control, allowing administrators to manage warehouses and view system-level analytics while users manage assigned warehouses and resolve their own alerts.

The automated sensor simulator is a key component of the project. It generates virtual sensor readings for temperature, humidity, and door status, allowing the alert detection workflow to be tested without physical IoT hardware. The alert engine successfully detects unsafe conditions such as high temperature, high humidity, and open door status.

Overall, the project demonstrates practical implementation of FastAPI, SQLAlchemy, SQLite, Jinja2 templates, role-based authentication, automated simulation, database design, alert generation, analytics, and testing. The system can be extended in the future with real IoT sensors, WebSocket-based real-time updates, cloud deployment, SMS/email notifications, and advanced compliance reporting.

References

[1] FastAPI Documentation, *FastAPI Framework for Building APIs with Python*. Available at: https://fastapi.tiangolo.com/

[2] SQLAlchemy Documentation, *Python SQL Toolkit and Object Relational Mapper*. Available at: https://docs.sqlalchemy.org/

[3] SQLite Documentation, *SQLite Database Engine*. Available at: https://www.sqlite.org/docs.html

[4] Jinja Documentation, *Jinja Template Engine*. Available at: https://jinja.palletsprojects.com/

[5] Uvicorn Documentation, *ASGI Web Server for Python*. Available at: https://www.uvicorn.org/